NUST CHIP DESIGN CENTRE

C / C++ Programming

Cache Simulator Design

Name	*Waleed Amir*
Instructor	*Sharjeel Khilji*
Date	*22th September 2025*

1. In-Lab Tasks: (*Write your lab task & screenshots here*)

i. Result:

```
cc@ncdc-0101:~/Dow
File  Edit  View  Search  Terminal  Help
[cc@ncdc-0101 cachelab-handout]$ make all
gcc -g -Wall -Werror -std=c99 -m64 -g -o csim csim.c cachelab.c -lm
[cc@ncdc-0101 cachelab-handout]$ ./csim-ref -s 4 -E 1 -b 4 -t traces/yi.trace
hits:4 misses:5 evictions:3
[cc@ncdc-0101 cachelab-handout]$ ./csim -s 4 -E 1 -b 4 -t traces/yi.trace
hits:4 misses:5 evictions:3
[cc@ncdc-0101 cachelab-handout]$ ./csim-ref -s 4 -E 1 -b 4 -t traces/yi2.trace
hits:16 misses:1 evictions:0
[cc@ncdc-0101 cachelab-handout]$ ./csim -s 4 -E 1 -b 4 -t traces/yi2.trace
hits:16 misses:1 evictions:0
[cc@ncdc-0101 cachelab-handout]$ ./csim-ref -s 4 -E 1 -b 4 -t traces/trans.trace
hits:211 misses:27 evictions:18
[cc@ncdc-0101 cachelab-handout]$ ./csim -s 4 -E 1 -b 4 -t traces/trans.trace
hits:211 misses:27 evictions:18
[cc@ncdc-0101 cachelab-handout]$ ./csim-ref -s 4 -E 1 -b 4 -t traces/long.trace
hits:257595 misses:29369 evictions:29353
[cc@ncdc-0101 cachelab-handout]$ ./csim -s 4 -E 1 -b 4 -t traces/long.trace
hits:257595 misses:29369 evictions:29353
[cc@ncdc-0101 cachelab-handout]$ ./csim-ref -s 4 -E 1 -b 4 -t traces/debug.trace
hits:0 misses:10 evictions:9
[cc@ncdc-0101 cachelab-handout]$ ./csim -s 4 -E 1 -b 4 -t traces/debug.trace
hits:0 misses:10 evictions:9
[cc@ncdc-0101 cachelab-handout]$ ./csim-ref -s 4 -E 1 -b 4 -t traces/dave.trace
hits:2 misses:3 evictions:0
[cc@ncdc-0101 cachelab-handout]$ ./csim -s 4 -E 1 -b 4 -t traces/dave.trace
hits:2 misses:3 evictions:0
[cc@ncdc-0101 cachelab-handout]$ S
```

Conclusion:

Working on this lab gave me a much clearer understanding of how cache memories actually function beyond the theory in textbooks. Writing my own simulator forced me to think carefully about set indices, tags, and block offsets, and how they all come together when a processor tries to fetch data. Implementing the LRU policy was tricky at first, but once I got it working, it was rewarding to see the hit, miss, and eviction counts match the reference simulator. I also learned the importance of dynamic memory allocation and structuring data in C, since the cache had to work for arbitrary sizes and associativities. What stood out to me most was realizing how even small design choices in memory systems can drastically impact performance.